

SAFESCALE

Un système hybride multi-agents et multi-objectifs pour la planification SAFe PI Planning

Optimisation NSGA-II, Critical Chain et HITL conditionnel

Gabriel Leulmi

Mémoire de Master, 2025-2026

Mai 2026

Résumé

La planification d'un *Program Increment* (PI, un cycle de livraison de huit à douze semaines) dans le cadre Scaled Agile Framework [22] est un **problème d'ordonnement sous contraintes de ressources** : on doit décider quoi faire, quand, et par quelle équipe, en respectant les compétences disponibles, les dépendances et les délais business. Ce type de problème (appelé RCPSP dans la recherche opérationnelle [21, 20]) est **non polynomial** : le nombre de plannings possibles explose si vite avec la taille du projet qu'aucun ordinateur ne peut les tester tous. La planification repose donc aujourd'hui sur l'expérience humaine. Le *Release Train Engineer* (RTE, facilitateur du train de livraison) anime le rituel, mais plusieurs autres rôles SAFe interviennent : *Scrum Master*, *Product Owner* (PO), *Chief Product Owner* (CPO), *Engineering Delivery Managers* et les *Development Teams* qui fournissent estimations et *refinement*. Les outils du marché servent à saisir et visualiser ces échanges, pas à explorer plusieurs scénarios candidats ni à les évaluer sur plusieurs critères. Ce mémoire présente **SAFESCALE**, un prototype d'aide à la décision qui assemble trois éléments. D'abord une optimisation par **algorithme génétique multi-objectifs** (NSGA-II [24, 8], qui fait évoluer une population de plannings candidats sur plusieurs générations), étendue à six objectifs dont une **tardivité pondérée par WSJF** (le retard de chaque feature en jours, multiplié par sa valeur business), inspirée d'un cas industriel multi-projets [15], avec une **recherche locale** qui peaufine les meilleurs plannings (approche « mémétique » [4]) et des plannings de départ construits par des heuristiques métier (inspirés d'INSGA-II [1, 2]). Ensuite une architecture multi-agents *orchestrée* en cinq étapes, avec un humain dans la boucle de manière **conditionnelle** (sollicité uniquement lorsqu'un problème est détecté) et une trace explicite des décisions. Enfin un modèle de risque déterministe inspiré de la *Critical Chain* de Goldratt et de la loi de Kingman [25], qui insère des marges de temps (*buffers*) après les *stories* les plus risquées. Le système tient compte du ratio durée-effort/durée-calendaire (un story point est rarement exécuté en autant de jours qu'il vaut), de la spécialisation des équipes (*one team owns one feature* [22]) et des goulots de capacité par compétence. Il est instancié sur un jeu de données synthétique reproductible (un ART, neuf équipes, quatre sprints de trois semaines, ≈ 135 *stories*) dont les dimensions sont inspirées d'un entretien avec un Product Owner.

Mots-clés. SAFe ; PI Planning ; RCPSP ; NSGA-II mémétique ; système multi-agents orchestré ; HITL conditionnel ; *Critical Chain* ; loi de Kingman ; tardivité pondérée WSJF.

1 Introduction

1.1 Contexte

Le *Scaled Agile Framework* (SAFe) définit la planification d'un *Agile Release Train* (ART, voir annexe A) regroupant 5 à 15 équipes et 50 à 125 personnes, organisée autour d'une cérémonie centrale, le **PI Planning**, sur une à deux journées pour planifier les huit à douze semaines suivantes [22]. Ce rituel mobilise plusieurs rôles dont les responsabilités sont précisément distribuées :

- le *Release Train Engineer* (RTE) anime le rituel et arbitre les dépendances entre équipes ;
- le *Chief Product Owner* (CPO) priorise les *features* de son périmètre et les répartit entre équipes ;
- chaque *Product Owner* (PO) porte le contenu de son équipe et présente ses propositions ;
- le *Scrum Master* (SM) facilite la cérémonie côté équipe ;
- les *Engineering Delivery Managers* (EDM) et les autres parties prenantes acceptent ou refusent les en-

gagements proposés ;

- les *Development Teams* produisent les estimations et le *refinement* qui alimentent toutes les décisions.

Aucun de ces rôles ne dispose aujourd'hui d'un outil pour explorer plusieurs scénarios candidats et les évaluer quantitativement ; les arbitrages se font à l'œil, sur la base de l'expérience.

1.2 Motivation

Formellement, ce que doit faire le RTE relève du *Multi-Skill Resource-Constrained Multi-Project Scheduling Problem* (MS-RCMPSP), une extension du RCPSP [21] prouvée NP-difficile dès 60 activités [20]. « Optimiser » signifie ici trois choses concrètes : (i) générer plusieurs scénarios de PI candidats au lieu d'un seul, (ii) les évaluer simultanément sur plusieurs critères, et (iii) recommander un compromis tout en gardant la décision finale chez l'humain. Cette complexité n'est pas adressée par les outils existants, qui restent centrés sur la saisie et la visualisation des artefacts SAFe. La recherche en optimisation a pourtant beaucoup travaillé

sur des problèmes voisins : les variantes hybrides de NSGA-II [4, 5] et NSGA-III [10, 12] donnent de bons résultats sur des instances de RCPSP et d'ordonnement d'atelier. L'objet de ce travail est de transposer ces approches au domaine SAFE, avec ses spécificités propres (WSJF, sprints, équipes spécialisées, statut de *readiness*).

1.3 Contributions

Le mémoire propose six contributions **originales** du système SAFESCALE, chacune transposant ou étendant un résultat publié dans la littérature au contexte spécifique du PI Planning SAFE :

- C1.** Formulation NSGA-II à six objectifs dont une tardivité pondérée par WSJF (somme des retards de chaque feature multipliés par sa valeur business), transposition au contexte SAFE d'un objectif validé sur un cas industriel multi-projets [15].
- C2.** **Six seeds d'initialisation** dont un *ratio WSJF / deadline urgency* adapté de la stratégie INSGA-II [1, 2].
- C3.** **Recherche locale mémétique** sur le front 0, inspirée du Memetic NSGA-II [4].
- C4.** Architecture **multi-agents orchestrée** en cinq étapes avec HITL *conditionnel*, dans la lignée des architectures multi-agent + NSGA-II [7].
- C5.** **Critical Chain Buffer** par *story*, fondé sur Goldratt 1997 et la loi de Kingman [25], avec score de risque déterministe à 5 facteurs pondérés.
- C6.** **Pipeline complet end-to-end** avec dataset synthétique reproductible calibré sur un ART réel.

1.4 Plan du mémoire

La section 2 formalise la problématique et précise le **périmètre** du prototype. La section 3 synthétise l'état de l'art en cinq familles. La section 4 décrit le modèle et les paramètres clés. La section 5 présente l'architecture du flow. La section 6 détaille NSGA-II. La section 7 synthétise les décisions de conception et leurs sources. La section 8 recense les problèmes de logique du flow et les corrections apportées. La section 9 décrit le protocole de test et les premiers résultats. La section 10 présente les retours qualitatifs sur la solution. La section 11 expose les limites et ce qui n'est pas pris en compte. La section 12 liste les points à explorer. La section 13 conclut. Deux annexes (A et B) fournissent un glossaire des sigles et des termes techniques.

2 Problématique

2.1 Le processus PI Planning existant

Avant de modéliser le problème, il faut comprendre comment un PI Planning se déroule *aujourd'hui*, sans outil d'optimisation. La description suivante est issue d'un entretien avec un *Product Owner* (PO) opérant sur un ART réel.

Avant le PI Planning. La préparation s'étale sur plusieurs semaines et mobilise plusieurs rôles :

- Le **CPO** (*Chief Product Owner*) **priorise les features** de son périmètre et s'assure qu'elles peuvent atteindre l'état *ready* : il organise pour cela des **ateliers de co-ingénierie** avec les équipes.
- La **durée des epics** est estimée en amont afin de répartir le travail sur les différents PI à venir.
- Le CPO **répartit les features par équipe** ; chaque équipe indique ensuite lesquelles elle est capable d'adresser. Il arrive qu'un epic soit **découpé en features réparties sur plusieurs équipes**.
- Une fois les features allouées, en **pré-PI Planning**, chaque PO prépare le périmètre de son équipe et recense les sujets à traiter.

Pendant le PI Planning. La cérémonie elle-même suit un déroulé bien établi :

- Le **Scrum Master** gère l'animation, le **PO** apporte la matière des discussions.
- On **liste les features** retenues pour le PI.
- Chaque équipe réalise un **poker planning** après le découpage des features en *stories*, en visant une taille de **8 story points maximum par story**.
- Les équipes **proposent les features** qu'elles se déclarent capables d'adresser. C'est à ce stade que le statut de *readiness* entre en jeu : si une équipe identifie des **incertitudes** sur certains items à développer (spécifications floues, dépendance externe non confirmée, etc.), elle ne se *commit* pas dessus pour le PI à venir, et peut proposer de les **repousser au PI suivant**.

Pendant l'exécution. La planification reste **volontairement générale** : à chaque sprint, les équipes tiennent un **meeting de réajustement** pour corriger le plan en fonction de la réalité du terrain.

Limites de ce processus. Ce déroulé est mature et éprouvé, mais il reste entièrement **manuel** : la répartition features→équipes, la détection des surcharges et des conflits de dépendances, l'arbitrage entre priorités concurrentes reposent sur l'expérience humaine. Aucune **exploration algorithmique** de scénarios alternatifs n'est possible, et aucune **évaluation quantitative multi-critères** n'assiste la décision. C'est précisément cet écart que SAFESCALE cherche à combler, en s'insérant comme une aide à la décision **sans modifier les rôles existants** (CPO, PO, Scrum Master, RTE).

2.2 Le PI Planning comme MS-RCMPSP

Formellement, le PI Planning est une instance de MS-RCMPSP combinant cinq extensions du RCPSP classique [21, 20] :

- *multi-projet* : plusieurs initiatives partagent les mêmes ressources [17] ;
- *multi-compétences* : ressources à plusieurs *skills* [16] ;
- *multi-objectifs* : six critères à optimiser simultanément, *makespan*, risque de retard, déficit de *buffer*, surcharge des ressources, violation de capacité par

- équipe et sprint, tardivité pondérée par WSJF, détaillés section 6 [8, 20];
- *contraintes d'équipe* : *one team owns one feature* [22];
- *dépendances inter-features* qui se propagent jusqu'aux *stories* [15].

2.3 Cinq familles de conflits récurrents

L'entretien mené avec un Product Owner, complété par notre modélisation, fait apparaître cinq familles récurrentes de conflits. Leur résolution manuelle, qui mobilise l'ensemble des rôles SAFe (RTE, CPO, PO, Scrum Master, EDM et équipes en dépendance), est ce que l'on cherche à instrumenter :

1. **CAPACITY_VIOLATION** (équipe au-delà de sa capacité);
2. **DEPENDENCY_BREACH** (*story* avant son prédécesseur);
3. **FEATURE_OVERFLOW** (features que l'ART ne peut pas absorber dans le PI courant et qui doivent être reportées au PI suivant);
4. **DEADLINE_RISK** (*feature* après sa deadline);
5. **LOAD_IMBALANCE** (>30 % d'écart entre équipes).

Ces cinq conflits sont *interdépendants* : résoudre une surcharge crée souvent une violation de dépendance, qui crée un risque de *deadline* [15].

2.4 Question de recherche

Peut-on concevoir un système hybride combinant optimisation multi-objectifs, architecture multi-agents et modèle de risque, pour automatiser la génération et l'évaluation de scénarios de PI Planning SAFe, tout en conservant à l'humain un rôle décisionnel effectif ?

2.5 Périmètre du prototype

Le prototype se concentre sur un cas d'étude défini précisément, afin de garantir la *reproductibilité* et la lisibilité des résultats :

- **Un seul ART**, neuf équipes spécialisées (*backend, frontend, data, devops, mobile, qa, fullstack* fallback), conformément à la fourchette 5-15 équipes par ART [22].
- **Un PI de quatre sprints de trois semaines** (60 jours ouvrés), format relevé lors de l'entretien PO et cohérent avec la fourchette 8-12 semaines [22].
- **Environ 57 ressources** (5-7 par équipe), 9 epics, ≈2 features par epic (≈18 features), 5 à 10 *stories* par feature (≈135 *stories*). La *story* est l'unité de planification : il n'y a pas de niveau *tâche*, conformément au processus réel observé (section 2.1).
- **Conversion effort → durée** : la valeur empirique varie d'une équipe à l'autre, typiquement entre **1 et 2 jours ouvrés par story point**. On utilise **1 SP = 1,7 jour** comme valeur pivot, en autorisant un tirage par équipe dans l'intervalle [1,0 ; 2,0].
- **Données synthétiques** générées avec une *seed* fixe (*seed*=42), garantissant la reproductibilité intégrale des expérimentations sans accès à des données réelles confidentielles.
- **Mode batch** : on planifie l'intégralité d'un PI en une session de bout en bout, déclenchée par le RTE. « Batch » ne signifie pas « sans interaction » : les cinq étapes du flow comportent des points HITL condition-

nels où le RTE intervient en cours d'exécution (cf. section 5). C'est le *replanning en flux continu* entre deux PI (par exemple en réaction à un événement extérieur) qui est hors périmètre.

Ce périmètre fixe le cadre du travail. Les éléments qui en sortent (données réelles Jira, multi-ART simultané, *replanning* temps réel, apprentissage par renforcement) sont explicitement reportés en limites (section 11) et en perspectives (section 12).

3 État de l'art

L'état de l'art est organisé en cinq familles couvrant les axes mobilisés par le système : algorithmes évolutifs multi-objectifs, extension à plus de trois objectifs, RCPSP hybride et multi-projets, pratiques officielles SAFe, et intégration du machine learning dans les MOEA.

3.1 Famille 1, NSGA-II et ses variantes

NSGA-II [24] reste, vingt ans après sa publication, « de loin la *métaheuristique* la plus populaire pour les problèmes d'optimisation multi-objectifs » [8]. Trois axes d'amélioration récents structurent notre contribution :

Axe 1, Initialisation ciblée. *INSGA-II* [1] démontre empiriquement sur 24 instances qu'une initialisation aléatoire dégrade fortement la convergence ; les stratégies adossées aux objectifs (WSJF, deadline-first, chemin critique, load-balance) améliorent significativement le taux d'optimisation [1, 2]. Notre seed C2 « WSJF / deadline urgency » prolonge cette idée en combinant valeur business et pression temporelle.

Axe 2, Hybridation mémétique. NSGA-II progresse par « grands sauts » aléatoires (croisements et mutations) : il explore très large, mais ne *peaufine* jamais une solution. Il peut passer juste à côté d'un optimum sans s'y arrêter. L'idée mémétique consiste à ajouter à NSGA-II une seconde mécanique complémentaire, la *recherche locale greedy* : on prend les meilleures solutions du moment et on leur applique des **petites retouches ciblées** (déplacer une story d'une équipe à une autre, inverser l'ordre de deux stories), en ne gardant que celles qui améliorent un objectif. Le tableau ci-dessous illustre la différence :

Sans recherche locale	Avec (mémétique)
On explore en grand au hasard, mais on ne polit aucune solution.	On explore en grand au hasard, puis on polit les meilleures par petites modifications.
Bonne couverture, qualité moyenne.	Bonne couverture <i>et</i> qualité fine.

Memetic NSGA-II [4] applique cette recette aux meilleurs individus du front 0 ; sur des problèmes d'ordonnancement d'atelier (*flexible job shop*), les auteurs obtiennent des fronts de qualité supérieure à ceux d'un solveur exact, pour un temps de calcul bien moindre. C'est la base de notre contribution C3.

Axe 3, NSGA-II + Q-Learning. Dans NSGA-II classique, le **moment** où chaque tâche démarre est choisi par des règles *fixes* (« le plus tôt possible une fois les dépendances satisfaites », par exemple). L'idée de cet axe est de remplacer cette règle figée par une **politique apprise** : un agent d'*apprentissage par renforcement* essaie différentes dates de début sur de nombreux plannings, reçoit un retour « c'était une bonne ou une mauvaise idée » selon la qualité du planning final, et *apprend* progressivement quelles dates marchent le mieux. [5] formalise cela comme un *processus de décision markovien*, résolu par l'algorithme *Q-Learning*. Le tableau ci-dessous illustre la différence :

Sans Q-Learning	Avec (NSGA-II + Q-Learning)
Dates de début choisies par une règle fixe, identique sur tous les plannings.	Dates de début choisies par une politique <i>apprise</i> sur des centaines de plannings antérieurs.
Pas besoin d'entraînement, mais qualité limitée par la règle.	Coûteux à entraîner, mais meilleure qualité finale.

Les résultats publiés surpassent NSGA-II pur, PSO et ACO. C'est lourd à entraîner mais riche en information ; on l'identifie comme perspective P3.

Ce qu'on retient et ce qui manque : ces trois axes valident séparément l'initialisation ciblée, la recherche locale et l'apprentissage du timing, mais aucun de ces travaux ne traite un problème structuré comme SAFe (hiérarchie epic/feature/story, contrainte *one team owns one feature*, WSJF). Ils restent sur des bancs d'essai génériques de *job shop* ; la transposition au PI Planning n'est pas immédiate et constitue justement une partie de notre travail.

3.2 Famille 2, NSGA-III pour 5+ objectifs

Avec six objectifs, la *crowding distance* de NSGA-II devient moins efficace pour préserver l'uniformité du front [10]. NSGA-III remplace ce mécanisme par un système de **points de référence** [12, 11], et obtient de meilleurs fronts que NSGA-II sur les benchmarks RCPSP de [12].

Ce qu'on retient et ce qui manque : NSGA-III est clairement plus adapté à notre nombre d'objectifs, et c'est pourquoi il figure en perspective P2. Mais il introduit un paramétrage supplémentaire, le placement des points de référence, qui demande une étude à part entière. Commencer par NSGA-II, mieux documenté et plus simple à déboguer, est un choix assumé de prototype, pas une fin en soi.

3.3 Famille 3, RCPSP hybride et multi-projets

Le *Hybrid Genetic Algorithm* de référence [13] reste une base classique : c'est un *algorithme génétique* standard auquel on greffe un *repair-based decoding* (cf. section 6) pour rendre la *mutation* toujours valide. [14] introduit le concept de *dense genes*, ce sont les gènes qui « saturent » l'utilisation d'une ressource (proches du chemin critique). L'idée est d'utiliser ces gènes denses comme *points d'ancrage* pendant le *croi-*

sement, plutôt que de mélanger les chromosomes au hasard : les bonnes parties d'une solution sont préservées. Sur les cas multi-projets industriels, [15] a inspiré notre 6^e objectif : la tardivité pondérée par WSJF (transposition de la *weighted tardiness* pondérée par criticité business). Pour la durabilité, [17] adapte NSGA-II au multi-projet, et [16] introduit MOFOA-HS pour les profils flexibles de ressource. Pour l'incertitude des durées, [18] convertit l'incertain (optimiste/pessimiste/probable) en objectifs ; [19] modélise les coûts variables dans le temps. La revue [20] conclut que l'hybridation domine généralement les algorithmes purs.

Ce qu'on retient et ce qui manque : cette famille fournit les briques techniques (*décodage*, croisement, objectif de tardivité) mais raisonne en « activités » abstraites. Elle ignore deux réalités du PI Planning : le découpage en sprints à capacité fixe, et le rôle d'un décideur humain qui tranche en fin de processus. Notre travail réutilise les briques mais les recompose autour de ces deux contraintes.

3.4 Famille 4, SAFe officiel et flow lean

La source officielle SAFe [22] fournit les paramètres normatifs : 5-15 équipes par ART, 50-125 personnes, 8-12 semaines par PI, ROAM board pour les risques. Reinertsen 2009 [25] formalise la relation entre utilisation et *lead-time* (loi de Kingman) : au-delà de 80 % d'utilisation, le lead-time devient super-linéaire. C'est la base de notre buffer 80 % bloquant (DEC-016) et de la valeur pivot 2.0 du *span_factor*.

Ce qu'on retient et ce qui manque : SAFe décrit *quoi* faire (les cérémonies, les rôles, les artefacts) mais pas *comment* optimiser le contenu d'un PI. La documentation officielle reste volontairement méthodologique et n'offre aucun outil quantitatif. C'est précisément l'espace que ce mémoire cherche à occuper.

3.5 Famille 5, ML + algorithmes évolutifs

Cette famille combine **apprentissage automatique** (ML) et **algorithmes évolutifs multi-objectifs** (MOEA), la catégorie dont NSGA-II fait partie. L'idée commune : utiliser un modèle ML, appris sur l'historique des projets, pour orienter l'optimisation vers des solutions à *risque* prédit plus faible. Le *survey* [23] en donne un panorama. *ENSGA* (*Enhanced NSGA-II, idem*) est un exemple : il glisse un modèle ML qui prédit la fatigue ou le risque des opérateurs dans la fonction objectif, et optimise à la fois la durée totale (*makespan*) et ces critères humains. Le multi-agent NSGA-II [7] traite quant à lui les problèmes à beaucoup d'objectifs en *répartissant* le calcul de la *crowding distance* (cf. section 6) entre plusieurs agents pour mieux couvrir le front.

Ce qu'on retient et ce qui manque : ces travaux montrent qu'un modèle de risque peut entrer dans la fonction objectif d'un MOEA. En revanche, ils mobilisent souvent du ML appris sur des historiques volumineux. Faute d'un tel historique, on a délibérément choisi un score de risque *déterministe* (équation 7), moins ambitieux mais reproductible et explicable, un compromis assumé.

3.6 Synthèse du positionnement

Prises séparément, ces cinq familles couvrent chacune un aspect du problème; aucune ne les réunit pour le PI Planning SAFe. C'est cette combinaison qui constitue l'apport du mémoire, résumé table 1 : (i) NSGA-II multi-objectifs adapté SAFe avec mémétique et seed *urgency*, (ii) MAS orchestré avec HITL conditionnel et trace, (iii) Critical Chain Buffer au niveau *story*.

Dimension	Existant	SAFESCALE
Optim.	RCPSP mono-obj.	NSGA-II 6-obj. + LS
MAS	BDI / centralisé	Orchestré + HITL
Risque	Heuristique	Critical Chain + score
Domaine	Ordonn. générique	SAFe (WSJF, sprints)
HITL	Final seul	Conditionnel par étape

TABLE 1 – Positionnement comparatif de SAFESCALE.

4 Modèle et données

4.1 Hiérarchie SAFe modélisée

SAFe distingue deux axes qu'il faut éviter de confondre : un axe d'**organisation** (qui travaille avec qui) et un axe de **décomposition du travail** (qui est découpé en quoi). Notre modèle reflète les deux séparément :

organisation : $\text{ART} \rightarrow \text{Team} \rightarrow \text{Ressource}$

décomposition : $(\text{Initiative}) \rightarrow \text{Epic} \rightarrow \text{Feature} \rightarrow \text{Story}$

Le niveau *Initiative* (entre parenthèses) existe en SAFe au-dessus de l'epic comme grand thème stratégique; on le mentionne pour la complétude mais on ne le modélise pas, faute d'effet sur la planification d'un seul PI. Notre modèle s'arrête à la *story* sur l'axe de décomposition, l'unité réellement estimée et planifiée dans le processus observé (section 2.1).

S'y ajoutent les compétences (*Skill*), les dépendances de *features* et de *stories*, et les artefacts de planification (*PIPlanning*, *StorySchedule*, *Scenario*, *DecisionTrace*). Conformément au processus réel, il n'y a **pas de niveau tâche** : la *story* est le grain le plus fin, estimée en *story points* (SP) lors du poker planning, avec un plafond de 8 SP par story.

4.2 Conversion effort, durée et étalement

L'effort d'une *story* est exprimé en *story points*. En pratique, un story point représente entre 1 et 2 jours-effort selon l'équipe; on retient une valeur pivot de 1,7 jour/SP, modulable par équipe pour refléter cette variabilité :

$$d_s = SP_s \times \alpha_{\text{team}(s)}, \quad \alpha_{\text{team}} \in [1,0; 2,0], \quad \alpha_{\text{défaut}} = 1,7 \quad (1)$$

Mais une story estimée à d_s jours-effort n'est jamais exécutée sur d_s jours calendaires consécutifs : réunions récurrentes, *context-switching*, blocages tech-

niques étalent le travail. On modélise cela par un **facteur d'étalement** :

$$\text{calendar}(s) = \lfloor d_s \cdot \text{span_factor}_{\text{team}(s)} \rfloor \quad (2)$$

Le *span_factor* a une valeur pivot de 2.0, mais il n'est pas identique pour toutes les équipes : le générateur tire pour chaque équipe une valeur dans l'intervalle $[1,8; 2,2]$. Cette valeur pivot de 2.0 correspond à la loi de Kingman appliquée à environ 50 % d'utilisation (lead-time $\approx 2 \times$ touch time) [25]; l'écart $\pm 0,2$ traduit le fait qu'une équipe expérimentée subit moins d'étalement (borne basse) qu'une équipe junior (borne haute).

4.3 Spécialisation des équipes

Chaque équipe a une spécialisation parmi sept catégories (*backend*, *frontend*, *data*, *devops*, *mobile*, *qa*, *full-stack*), conforme à la notion SAFe d'*Agile Team* [22]. La distribution des compétences suit une règle 80/20 : 80 % des skills d'une ressource viennent du cœur de spécialité de son équipe, 20 % peuvent déborder. Les compétences rares (*data*, *mobile*, *devops*) sont *exclusives* aux équipes correspondantes, ce qui doit permettre de provoquer un goulot capacité observable (cf. section 11).

4.4 Dataset synthétique de référence

Le générateur produit, pour *seed*=42, un dataset reproductible aux dimensions relevées sur un ART réel (section 2.1) :

Entité	Quantité
ART	1
Équipes spécialisées (5-15 SAFe)	9
Ressources (5-7 / équipe)	≈ 57
Epics	9
Features (≈ 2 / epic)	≈ 18
Stories (5-10 / feature, ≈ 8 SP)	≈ 135
PI : 4 sprints $\times$ 3 semaines	60 j

5 Architecture et flow

5.1 Vue technique en trois couches

L'application est structurée en **frontend React + TypeScript** (Tailwind, Recharts), **backend Python + Django + DRF**, et **base PostgreSQL** (figure 1). Un orchestrateur central (*discussion_flow.py*) pilote des agents spécialisés via un protocole synchrone requête/réponse, à la manière du multi-agent NSGA-II proposé par [7]. Les LLM interviennent exclusivement comme **traducteurs** (*rationale* en langage naturel), jamais comme décideurs; ils sont *mockables* pour la reproductibilité des tests.

5.2 Flow en cinq étapes

Le flow est résumé figure 2 :

1. **Préparation** (*readiness*) : classement de chaque feature en READY / BORDERLINE / NOT_READY, conforme au principe de « backlog refinement » [22]. Pour chaque équipe, le *Product Owner* reste le propriétaire du backlog; il s'appuie sur de nombreux autres acteurs (CPO, EDM, Scrum Master, équipes en dépendance) qui influencent les priorités et l'état de préparation. HITL si des features restent non prêtes.

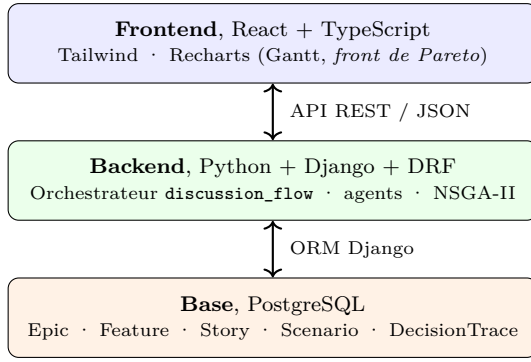


FIGURE 1 – Architecture en trois couches de SAFESCALE.

2. **Dépendances** : on construit le graphe orienté des dépendances entre features et entre stories (le DAG), puis on le parcourt en **profondeur d'abord** (*Depth-First Search*, DFS) pour deux usages : (i) détecter les cycles de dépendance, qui rendraient la planification impossible ; (ii) calculer le *chemin critique*, la plus longue chaîne de dépendances qui fixe la durée minimale du PI [15].
3. **Capacité & allocation** : chaque équipe construit son plan capacitaire. **HITL bloquant si une équipe dépasse 80 % de sa capacité** [25].
4. **Planification** : NSGA-II (encodage *story* → ressource) produit un front de Pareto, dont on extrait $K = 5$ scénarios **diversifiés**. La diversification est évaluée par la *distance de Hamming* sur l'assignation feature → équipe : deux scénarios sont d'autant plus différents qu'ils affectent plus de features à des équipes distinctes. La sélection *farthest-first* (cf. contribution 6) garantit que chaque nouveau scénario présenté est le plus éloigné possible des précédents, ce qui évite de proposer cinq variantes du même compromis.
5. **Choix du scénario : décision RTE explicite**, inspirée du *max-regret* de [5] et du ROAM board SAFe [22].

5.3 HITL conditionnel

Plutôt que de demander à chaque étape « veux-tu continuer ? », chaque agent décide localement s'il peut trancher seul ou s'il doit ouvrir un point HITL. Cette logique « conditionnelle » est ce qui différencie SAFESCALE de la plupart des outils SAFe existants où l'humain est soit absent, soit sollicité partout.

5.4 Traçabilité (DecisionTrace)

Chaque décision est persistée dans un modèle *DecisionTrace* (question posée, décision prise, justification, alternatives écartées, *snapshot* des inputs, diff des outputs, directives humaines, score de confiance, *prompt* et réponse LLM). La traçabilité multi-niveaux est nécessaire pour la défense soutenue dans la revue *Comprehensive Review* sur NSGA-II combinatoire [8].

6 NSGA-II : formulation et implémentation

6.1 Encodage du chromosome

Un individu est une liste $\mathbf{x} = (x_1, \dots, x_n)$ où $x_i = (story_id_i, resource_id_i)$: chaque gène affecte une *story* à une ressource. L'ordre des gènes encode l'ordre d'exécution. La validité topologique est garantie au croisement et à la mutation par un *repair-based decoding* - mécanisme qui, après chaque opération génétique, parcourt le chromosome et réordonne automatiquement les gènes en conflit (dépendance violée, ressource déjà occupée) au lieu de rejeter la solution, hérité de [13].

6.2 Décodage

$$earliest_i = \max \{ end_p \mid p \prec i \} \quad (3)$$

$$start_i = \max (earliest_i, avail_{r(i)}) \quad (4)$$

$$end_i = start_i + calendar(s_i) \quad (5)$$

$$avail_{r(i)} \leftarrow end_i + buffer_i \quad (6)$$

où $calendar(s_i)$ est la durée calendaire de la *story* s_i (équations 1–2) et le buffer $buffer_i$ correspond au *Critical Chain Feeding Buffer* défini ci-après.

6.3 Critical Chain Buffer

Conformément au principe de Goldratt 1997 « retirer les marges individuelles et les regrouper en buffers ciblés sur les points de variance » (cité dans [18]), on calcule un score de risque déterministe pour chaque *story* :

$$risk(s) = 0,30 \cdot \tilde{c} + 0,20 \cdot \tilde{\sigma} + 0,20 \cdot \tilde{p} + 0,20 \cdot \mathcal{K}_{rare}(s) + 0,10 \cdot \tilde{d} \quad (7)$$

$$buffer(s) = \lfloor risk(s) \cdot 5 \rfloor$$

où $\tilde{c}, \tilde{\sigma}, \tilde{p}, \tilde{d}$ sont respectivement la complexité, le *span_factor*, le nombre de prédécesseurs et l'effort (en SP), normalisés ; $\mathcal{K}_{rare}$ vaut 1 si la compétence requise est *data*, *mobile* ou *devops* (catégories sous-représentées). Cette modélisation prolonge l'idée de [23], qui valide empiriquement l'intégration d'un modèle de risque dans la fonction objectif d'un MOEA, et celle de [18], qui convertit l'incertitude en objectifs.

6.4 Six objectifs (à minimiser)

Ces six objectifs **ne sont pas indépendants** : au contraire, ils se contredisent. Si on essaie de compresser le *makespan*, on parallélise plus, donc on surcharge les ressources et le risque monte. Si à l'inverse on garde de la marge pour baisser le risque, on allonge le *makespan* et certaines features peuvent finir après leur deadline. Il n'existe donc pas un seul planning qui soit le meilleur sur les six critères en même temps. C'est exactement pour ça qu'on utilise un *algorithme évolutif* multi-objectifs : au lieu de viser un optimum unique, NSGA-II construit un *front de Pareto*, c'est-à-dire un ensemble de compromis où chaque gain sur un critère se paie ailleurs.

Pondération. Pendant la recherche, NSGA-II **ne pondère pas** les objectifs : tous sont mis sur un pied d'égalité via la relation de dominance (équation 9). La

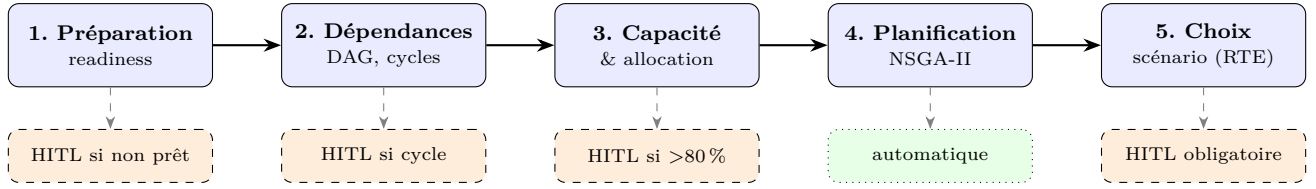


FIGURE 2 – Le flow en cinq étapes. L'étape 4 (NSGA-II) est entièrement automatique ; les étapes 1, 2 et 3 ouvrent un point HITL *conditionnel*, seulement si un problème est détecté, tandis que l'étape 5 requiert toujours une décision humaine explicite.

pondération arrive uniquement à la toute fin, quand il faut choisir un scénario « recommandé » parmi ceux du front. À ce moment-là, on calcule un *score composite* (équation 13) dont les poids w_j peuvent être ajustés par le RTE. Par défaut, les six poids valent 1/6 chacun.

Les six objectifs sont :

1. Makespan $C_{\max}$ [24].
2. Risque ML moyen $\frac{1}{n} \sum_i risk(s_i)$ [23].
3. Buffer déficit (marge manquante).
4. Surcharge ressource (parallélisme).
5. Violation de capacité par équipe et sprint [22].
6. **Tardivité pondérée WSJF** (contribution C1) :

$$f_6 = \frac{1}{n} \sum_i WSJF_i \cdot \max(0, end_i - deadline_i) \quad (8)$$

Transposition au contexte SFA de l'objectif *weighted tardiness* validé sur un cas industriel multi-projets [15].

6.5 Dominance et front de Pareto

Dominance de Pareto stricte [24] :

$$\mathbf{x} \prec \mathbf{y} \iff \forall j, f_j(\mathbf{x}) \leq f_j(\mathbf{y}) \wedge \exists k, f_k(\mathbf{x}) < f_k(\mathbf{y}) \quad (9)$$

Crowding distance [24] :

$$d_c(i) = \sum_{j=1}^M \frac{f_j(i+1) - f_j(i-1)}{f_j^{\max} - f_j^{\min}} \quad (10)$$

6.6 Six seeds intelligents

NSGA-II démarre avec une **population initiale** de plannings candidats, typiquement 40 à 60 individus. Si on génère ces plannings totalement au hasard, l'algorithme passe les premières générations à « rattraper son retard ». INSGA-II [1, 2] a montré qu'il est bien plus efficace d'**injecter quelques plannings construits par des heuristiques simples** dans la population initiale : l'évolution démarre alors près de « bonnes régions » de l'espace de recherche.

Nos six heuristiques de génération produisent chacune un planning initial complet en ordonnant les stories selon un critère métier : (1) WSJF descendant, valeur business d'abord ; (2) deadline ascendante, urgence d'abord ; (3) load-balancé, répartir la charge équitablement ; (4) chemin critique d'abord [15] ; (5) risque ML minimal d'abord [23] ; (6) **ratio WSJF / deadline urgency** (C2) qui combine valeur et pression temporelle :

$$urgency(f) = \frac{WSJF_f}{\max(1, deadline_f - now)} \quad (11)$$

Le reste de la population initiale est complété par des plannings aléatoires pour garder de la diversité.

6.7 Recherche locale mémétique (C3)

Toutes les 5 générations, recherche locale *greedy* sur les 15 % supérieurs du front 0 (réassignation de ressource, swap d'ordre), à la suite de [4]. On ne garde la mutation que si elle ne dégrade aucun objectif (test de non-dominance).

6.8 Sélection des K=5 scénarios diversifiés

Le front de Pareto contient souvent des dizaines de solutions non-dominées. On ne peut pas toutes les présenter au RTE, et choisir les cinq « meilleures » ne marche pas : on tombe vite sur cinq plannings qui se ressemblent tous, parce qu'ils sont voisins du même optimum. La revue [8] et le multi-agent NSGA-II [7] insistent sur l'importance d'une **bonne couverture** du front, pas seulement sur la qualité brute des solutions retenues.

Mesurer la différence entre deux plannings. On utilise la *distance de Hamming*, qui compte le nombre de positions où deux plannings diffèrent. Ici, chaque « position » correspond à une feature, et la valeur stockée est « quelle équipe l'a prise ». Deux plannings où toutes les features sont allouées aux mêmes équipes ont une distance de 0 ; deux plannings qui répartissent chaque feature à une équipe différente sont à distance maximale.

Choisir les 5 plannings les plus différents. On part du scénario recommandé par le score composite, puis on ajoute un à un les quatre suivants en appliquant une règle dite *farthest-first* : à chaque étape, on prend dans le front celui qui est le **plus loin** des scénarios déjà retenus. Cette règle s'écrit :

$$\mathbf{x}_{k+1} = \arg \max_{\mathbf{x} \in \mathcal{F}^0 \setminus \mathcal{S}_k} \min_{\mathbf{y} \in \mathcal{S}_k} d_H(\mathbf{x}, \mathbf{y}) \quad (12)$$

$\mathcal{F}^0$ est l'ensemble du front Pareto, $\mathcal{S}_k$ les k scénarios déjà choisis, et d_H la distance de Hamming. À chaque ajout, on garantit que le nouveau scénario est différent de tous les précédents, ce qui évite les cinq variantes du même compromis.

Score composite (recommandé). Pour désigner le scénario « recommandé » par défaut, on normalise chaque objectif entre 0 et 1 et on en fait une moyenne

pondérée :

$$score(\mathbf{x}) = \sum_{j=1}^6 w_j \cdot \frac{f_j(\mathbf{x}) - f_j^{\min}}{f_j^{\max} - f_j^{\min}} \quad (13)$$

$f_j^{\min}$ et $f_j^{\max}$ sont les bornes observées sur le front ; les w_j sont les poids des six objectifs (par défaut 1/6 chacun, ajustables par le RTE).

6.9 Conseil de retrait au step 3

Quand une équipe dépasse 80 % de charge [25], on calcule un score d'impact déterministe :

$$impact(f) = \frac{1}{WSJF_f \cdot (1 + downstream_blocks_f)} \quad (14)$$

Plus le score est bas, plus la *feature* est candidate au retrait (faible valeur business, peu de successeurs cascades). La cascade applique `on_delete=CASCADE` au niveau `Feature→Story` et signale les successeurs DAG potentiellement orphelins.

6.10 Détection de goulot capacité

$$demand(s) = \sum_{t: skill(t)=s} d_t \quad (15)$$

$$supply(s) = \sum_{r: s \in skills(r)} avail_r \cdot PI_{days} \quad (16)$$

$$gap(s) = \max(0, demand(s) - supply(s)) \quad (17)$$

7 Décisions de conception

Le tableau 2 synthétise les décisions de conception majeures avec leur justification. Deux sources ont nourri ces choix : la littérature scientifique récente d'une part, et d'autre part un **entretien semi-directif avec un Product Owner** opérant sur un ART SAFe. Cet entretien n'a pas valeur d'étude de terrain exhaustive, mais il a permis de cadrer des éléments qu'aucun article ne documente précisément, le format de PI à 4 × 3 semaines, la conversion empirique story points/jours, ou l'application stricte du buffer 20 %. Quand une décision s'appuie sur cet entretien, la mention *Entretien PO* l'indique dans le tableau.

8 Problèmes de logique du flow et solutions

Le flow décrit en section 5 n'a pas été conçu d'un seul jet. Sa première version était plus naïve, et plusieurs défauts de logique ne sont apparus qu'en faisant tourner le système de bout en bout sur le dataset de référence : un consensus qui se forme sur des chiffres trompeurs, une étape qu'on traverse sans s'en rendre compte, un optimiseur qui produit un plan absurde mais « valide ». Cette section rassemble ces défauts et la correction que j'ai apportée à chacun. Ce ne sont pas des bugs de code mais des erreurs de **conception du flow** ; je les présente sous la forme *symptôme* puis *correction*, en renvoyant à la décision DEC-XXX correspondante.

8.1 Agents équipes négociant sur des agrégats SP

Symptôme. La première version du flow faisait négocier les agents équipes sur des estimations agrégées

en *story points*. Conséquence : leurs arguments étaient flous, leurs refus de portfolio non explicables, et le swarm pouvait converger sur un consensus infaisable côté ressources et calendrier, un goulot interne (par exemple une ressource senior surchargée à 130 % sur le sprint 2) restait masqué derrière la moyenne SP.

Correction (DEC-004). Aligné sur l'approche en deux phases du Memetic NSGA-II [4] : on exécute d'abord une passe exploratoire globale qui produit des plannings concrets (assignation *story→ressource*, dates, charge par sprint), puis on donne ces plannings aux agents équipes pour qu'ils négocient sur données réelles. Un re-scheduling *warm-start* valide les contre-propositions avant convergence (DEC-005), conformément au schéma à deux temps de [13].

8.2 NSGA-II éclatant les features sur plusieurs équipes

Symptôme. Sans contrainte explicite, l'optimiseur produisait des plans techniquement excellents mais distribuant les *stories* d'une même *feature* sur 3-4 équipes différentes, violation flagrante du principe SAFe « *one team owns one feature* » [22]. Le processus réel admet ce découpage (un epic est parfois réparti en features sur plusieurs équipes, section 2.1) mais le traite comme exception négociée, jamais comme cas par défaut.

Correction (DEC-006). Pénalité *soft fort* (×100) sur chaque *story* éclatée hors de l'équipe majoritaire de sa *feature*. Une contrainte dure aurait planté le solveur sur des cas limites légitimes (*component teams* [22]) ; le *soft fort* laisse une soupape mais éteint le problème en pratique.

8.3 Buffer 20 % SAFe non respecté à l'étape 3

Symptôme. La règle empirique SAFe (issue de la loi de Kingman [25] : >80 % utilisation ⇒ lead-time super-linéaire) est qu'on ne doit pas dépasser 80 % de charge prévue. Or l'étape 3 laissait passer le flow vers NSGA-II tant que la charge restait ≤ 110 %. NSGA-II partait alors sur un portfolio infaisable et trouvait « les moins pires » compromis au lieu d'arbitrer en amont.

Correction (DEC-016). Seuil bloquant abaissé à 80 % par défaut : tant qu'au moins une équipe dépasse ce seuil ou refuse son portfolio, le flow reste en HITL à l'étape 3. La règle est explicitée dans un bandeau citant Reinertsen [25].

Nuance terrain. L'entretien PO a confirmé qu'en pratique **chaque équipe définit son propre flow**, et que le pourcentage de capacité réellement dédiable au PI Planning varie selon la maturité, la topologie et la **charge externe** de l'équipe (volume de support, afflux de *Problem Change Requests* suite à une campagne de recette, etc.). Le 80 % du buffer SAFe est donc une *valeur cible par défaut* et non une constante universelle. Le seuil bloquant est paramétrable par équipe dans le modèle de données ; il est exposé au RTE pour qu'il puisse abaisser ce seuil sur les équipes connues pour absorber beaucoup de travail hors PI.

8.4 Surcharge sans boucle de rééquilibrage

Symptôme. Quand l'étape 3 détectait une surcharge, le flow n'offrait **aucun mécanisme de rééquilibrage automatique** : la seule sortie possible était de

Réf.	Décision	Justification (sources académiques + observation terrain)
DEC-001	Spécialisation des équipes (enum + <i>fullstack</i> fallback)	[22] : un ART contient 5-15 équipes typées. <i>Entretien PO</i> : les ARTs réels sont organisés en équipes <i>back / front / data / devops / mobile / qa</i> avec une équipe transverse <i>fullstack</i> pour les sujets cross.
DEC-002	span_factor pivot 2.0, tiré dans [1,8; 2,2] par équipe	[25] loi de Kingman ($\approx 50\%$ utilisation $\Rightarrow$ lead-time $\approx 2\times$).
DEC-004	NSGA-II exploratoire global puis raffinement	[4] (deux phases mémétique) ; [13] (HGA en deux temps).
DEC-005	Re-scheduling <i>warm-start</i> des contre-propositions	[13] : schéma à deux temps. Valide les transferts de <i>features</i> proposés par les équipes avant la convergence du <i>swarm</i> .
DEC-006	Cohésion feature $\rightarrow$ équipe : pénalité $\times 100$	[22] : « one team owns one feature » ; [16] : profils flexibles avec coût. <i>Entretien PO</i> : l'éclatement d'une feature sur plusieurs équipes est traité comme exception négociée, jamais comme cas par défaut.
DEC-007	Diversité Pareto par farthest-first Hamming	[7] : crowding multi-agent ; [9] : opérateurs de congestion.
DEC-008	Modèle DecisionTrace runtime	[8] : revue insistant sur la traçabilité combinatoire.
DEC-010	PI 12 semaines $\times$ 4 sprints de 3 semaines	[22] (PI 8-12 sem.) ; <i>entretien PO</i> : format 4×3 semaines effectivement utilisé (cohérent avec la <i>cadence</i> trimestrielle des Business Owners).
DEC-014	Conseil IA + rééquilibrage HITL au step 3	[23] : intégration ML/MOEA pour aide à la décision ; [22] : ROAM board. <i>Entretien PO</i> : les RTEs effectuent une <i>capacity review</i> explicite et négocient les retraits feature par feature.
DEC-015	Choix scénario RTE explicite à l'étape 5	[5] : max-regret pour décision multi-acteurs ; [22] : revue Business Owner obligatoire. <i>Entretien PO</i> : la décision finale n'est jamais déléguée à un consensus mécanique, le RTE tranche en présence des Product Owners.
DEC-016	Buffer 80 % bloquant	[25] (Kingman) ; [22]. <i>Entretien PO</i> : la règle des 20 % de buffer (capacity load $\leq 80\%$) est appliquée strictement par les RTEs avant validation du PI.
DEC-018	Critical Chain Buffer par <i>story</i>	Goldratt 1997 (cité dans [18]) ; [23] : ML+MOEA.
C1	Tardivité pondérée WSJF (6 ^e obj.)	[15] : cas industriel multi-projets, <i>weighted tardiness</i> .
C2	Seed WSJF/deadline urgency	[1] INSGA-II ; [2] règles composites.
C3	Recherche locale mémétique	[4] : recherche locale <i>greedy</i> sur le front 0.

TABLE 2 – Décisions de conception majeures, avec leurs sources académiques et, pour les choix concernés, les éléments issus de l'entretien PO.

poursuivre vers NSGA-II avec une allocation infaisable. L'algorithme partait donc sur un portfolio intrinsèquement surdimensionné et produisait des compromis sous-optimaux par construction.

Correction (DEC-014). Introduction d'un conseil IA déterministe (équation 14) qui ordonne les *features* candidates par impact croissant, et d'une boucle de re-CNP (Capacity Negotiation Protocol) bornée par la règle des 80 %. Suivant l'esprit du *survey* [23] sur l'intégration ML/MOEA en aide à la décision multi-objectifs, la boucle ne se ferme qu'une fois la contrainte de capacité satisfaite. L'optimiseur reçoit alors un portfolio sain plutôt que de devoir compenser en aval.

8.5 Auto-finalisation court-circuitant la décision finale

Symptôme. L'étape 5 auto-finalisait sur consensus dès que les agents équipes s'accordaient sur deux stratégies préférentielles ou moins. Le *flow* sautait alors directement de l'étape 3 à un état « PI Planning terminé » sans avoir matérialisé un choix explicite : le scénario « recommandé » par le système était considéré comme accepté par défaut.

Correction (DEC-015). Étape 5 rendue toujours HITL par construction du flow. Inspirée du *max-regret*

de [5], l'orchestrateur calcule un consensus indicatif (majorité de votes, recommandé par le *score composite*, Pareto-optimal) mais ne déclenche jamais la transition vers l'étape 6 sans une décision explicite. Cela respecte la sémantique du HITL conditionnel : conditionnel à *un problème détectable algorithmiquement*, et non sur une décision business fondamentalement humaine [22].

Calage sur le déroulé réel d'un PI Planning. Le rituel observé chez l'interviewé se déroule sur deux journées avec une structure claire, qu'on cherche à reproduire dans l'outil :

- **Journée 1.** Chaque PO présente la proposition de son équipe ; les parties prenantes (RTE, CPO, EDM, équipes en dépendance) acceptent ou refusent. Un *partage de synthèse* en fin de J1 permet d'identifier les difficultés à anticiper pour J2.
- **Journée 2.** Les équipes affinent leurs engagements en intégrant les contre-propositions de la veille. Une *revue de validation finale* en fin de J2 acte le plan définitif.
- **Issue possible.** Il arrive que le plan *ne soit pas accepté* à la fin des deux jours : dans ce cas, d'autres réunions sont tenues au cours des premiers jours du PI lui-même pour faire converger les équipes et les parties prenantes vers un plan validé.

SAFESCALE modélise cette structure en deux phases côté front-end : les K scénarios sont d'abord présentés en mode *revue* (analogue à J1, chaque équipe vote, on identifie les points durs), puis le RTE effectue la *validation finale* (J2) à partir des votes et du conseil de l'évaluateur. L'absence de validation explicite en fin de J2 produit un état « PI Planning en attente d'arbitrage » plutôt qu'un état « terminé ».

8.6 Front Pareto donnant l'illusion du choix

Symptôme. Le front Pareto contenait quelques dizaines de solutions, dont beaucoup étaient des voisins d'un même optimum. Sélectionner $K = 5$ par score composite produisait des scénarios visuellement différents mais qualitativement identiques.

Correction (DEC-007). On force la **diversification** des scénarios proposés au RTE : au lieu de prendre les $K = 5$ meilleurs au score composite, on construit la liste un à un en prenant à chaque fois le scénario le plus différent de ceux déjà choisis. La distance utilisée est la *distance de Hamming* sur l'assignation *feature*→*équipe* (cf. section 6 pour le détail). À la suite de [7] (uniformité du front) et [9] (opérateurs de congestion).

8.7 Variance d'exécution et équipes en sous-activité

Constat terrain. La contrainte majeure observée aujourd'hui sur les ARTs reste la **surcapacité** : des équipes engagées au-delà de ce qu'elles peuvent absorber. Mais le retour PO décrit aussi le phénomène inverse — des équipes **en sous-activité** sur un PI, soit parce qu'elles ont une vélocité plus haute que prévu, soit parce que leur portefeuille s'est dégonflé entre la planification et l'exécution.

Symptôme côté optimiseur. Avec l'équation 2 seule, deux problèmes apparaissent :

- une équipe planifiée à 80 % voit son planning se compacter sur 2 sprints alors qu'elle dispose de 4, sans aucune marge pour la variance d'exécution ;
- l'optimiseur ne propose rien pour valoriser la capacité libérée par une équipe en sous-activité.

Correction (DEC-018) — pour la variance d'exécution. Insertion de **buffers Critical Chain** ciblés sur les *stories* à variance élevée (équation 7), conformément à Goldratt 1997 et au *survey* ML+MOEA [23]. Variante du *fuzzy RCPSP* [18] qui convertit l'incertitude en objectifs ; ici on la convertit en buffers locaux.

Piste pour la sous-activité (à explorer). Quand une équipe est sous-utilisée, deux leviers métier sont envisageables, en complément des buffers :

- **Anticiper des stories du PI suivant** : remonter dans le backlog des *features* prévues pour le PI $N+1$ et les ajouter de manière opportuniste si la compétence colle ;
- **Prendre en charge des stories d'autres équipes** surchargées, à condition que la spécialisation le permette (*cross-team support*, typiquement par une équipe *fullstack*).

Ces deux mouvements sont déjà partiellement modélisables dans SAFESCALE : la première s'inscrit dans le mécanisme de retrait/ajout de *features* de l'étape 3, la seconde dépend du modèle de spécialisation déjà décrit

en section 4. Leur intégration explicite est listée comme perspective P7 (section 12).

8.8 Goulot capacité par compétence non instrumenté

Symptôme. Un goulot sur une compétence rare (*ml_ops*, *mobile*, *devops*) ne ressort jamais comme signal isolé : la surcharge se manifeste à l'échelle équipe et noie l'origine. Le RTE retire des *features* « au hasard » sans savoir laquelle débloque le goulot.

Correction. Service `skill_bottleneck` qui calcule *demand*, *supply* et *gap* **par compétence**, identifie les équipes couvrantes, et pondère les *features* candidates au retrait par leur effort libéré sur la compétence en goulot. Cas extrême : compétence non couverte par aucune équipe $\Rightarrow$ bonus +5 au score de retrait des *features* qui la requièrent. Cette modélisation s'appuie sur la notion de *flexible resource profile* de [16].

9 Tests et résultats expérimentaux

Cette section sera complétée à l'issue des campagnes expérimentales. Elle présentera le protocole retenu, les métriques de qualité du front de Pareto (*hypervolume*, *spread*), la comparaison entre NSGA-II standard et ses variantes (seeds, mémétique), le comportement du flow étape par étape, ainsi que les résultats des tests automatisés (taux de réussite, couverture).

Deux protocoles sont prévus en complément :

- **Comparaison agent vs PO humain.** Sur un même dataset, on demande à un PO de construire un PI Planning à la main, puis on lance SAFESCALE. On mesure les écarts d'allocation *features*→*équipes*, d'ordre d'exécution et de KPI ; les divergences révèlent soit des problèmes de pondération des critères dans le système, soit des arbitrages qu'un humain n'aurait pas vus.
- **Étiquetage des scénarios.** Les K scénarios du front sélectionné (cf. section 10) seront automatiquement nommés selon des règles déterministes basées sur les KPI dominants (par exemple « makespan minimal », « risque le plus faible », « charge la mieux équilibrée »), de manière à ce que les étiquettes parlent directement au PO sans connaissance des algorithmes sous-jacents.

10 Retours sur la solution

10.1 Observations qualitatives

Les retours qui suivent sont **qualitatifs** : ils proviennent de sessions de démonstration menées en interne sur le dataset de référence, et non d'une évaluation chiffrée, celle-ci fera l'objet de la section 9 une fois les campagnes terminées.

Lisibilité du flow. Découper le processus en cinq étapes nommées (*Préparation*, *Dépendances*, *Capacité & allocation*, *Planification*, *Choix du scénario*) rend chaque arbitrage plus facile à suivre. On peut reprendre une session étape par étape et expliquer ce qui s'est passé, ce qui est utile pour la défense.

Effet du buffer 80 %. Tant que le seuil bloquant n'existait pas, l'étape 3 se traversait vite et sans vraie discussion. En la rendant bloquante, elle devient un moment de négociation : le RTE voit explicitement pour

quoi il doit retirer des *features*, puisque la règle est affichée.

Conseil de retrait. Le panneau de retrait au step 3 ordonne les *features* candidates par score d'impact (équation 14). En pratique, il met en avant des arbitrages peu intuitifs, par exemple qu'une *feature* à WSJF moyen sans *deadline* libère plus de marge qu'une *feature* à WSJF élevé mais à *deadline* serrée.

10.2 Cohérence du Pareto

La sélection *farthest-first* (DEC-007) produit des scénarios visiblement distincts : on peut leur donner des étiquettes parlantes (« rapide », « sûr », « économe »). C'est l'effet recherché, éviter de présenter au RTE cinq variantes du même compromis.

Comment ces étiquettes sont générées. Une fois les cinq scénarios sélectionnés, on regarde sur lequel des six objectifs chacun est le **meilleur du lot** et on en déduit son nom :

Meilleur sur...	Étiquette générée
makespan	« rapide »
risque ML	« sûr »
violation de capacité	« économe »
tardivité WSJF	« prioritaire »
buffer déficit	« avec marge »
surcharge	« équilibré »

Les noms sont volontairement **courts et métier** : le PO et le RTE peuvent les utiliser pour se parler (« je préfère le scénario rapide, mais avec un peu plus de marge ») sans connaître la formule mathématique qui a déterminé le classement. Si plusieurs scénarios sont meilleurs sur le même objectif, on tranche au second meilleur KPI, ce qui produit des étiquettes composées (« rapide-sûr »).

10.3 Reproductibilité

La *seed* fixe, le mode *mock* du LLM et la trace *DecisionTrace* permettent de rejouer une session à l'identique. C'est important ici : sans cela, on ne pourrait ni comparer deux versions du système, ni vérifier un résultat lors de la soutenance.

11 Limites et ce qui n'est pas pris en compte

Le périmètre du prototype a été précisé en section 2.5. Cette section recense ce qui en a été volontairement exclu, les limites algorithmiques connues, et les limites expérimentales du dataset de référence.

11.1 Hors-périmètre fonctionnel

- **Données réelles Jira** non encore ingérées : le passage en production nécessite l'adaptation au format Jira Agile@Scale ainsi que la gestion des autorisations.
- **Multi-ART** non supporté : la planification se fait sur un ART à la fois. Le multi-projet existe (épique → *feature*) mais pas le multi-ART simultané, traité dans la littérature [17].
- **Replanning temps réel** non traité : le système fonctionne en mode *batch*, le *replanning* continu en cours

de PI relève d'une autre classe de problème.

- **Apprentissage par renforcement** non implémenté : l'hybridation NSGA-II + Q-Learning de [5] reste à explorer.

11.2 Limites algorithmiques

- **NSGA-II avec *crowding distance*** : NSGA-III [10, 12] serait plus adapté pour 6 objectifs.
- **Pas de tampon global de fin de PI.** En l'état, on insère un petit tampon *après chaque story à risque* pour encaisser ses éventuels retards (c'est ce qu'on appelle un *feeding buffer*). La méthode de Goldratt préconise en plus un **gros tampon unique en toute fin de PI** (le *project buffer*), qui sert de filet de sécurité agrégé pour tout l'ART. On ne l'a pas encore mis en place : le RTE pilote donc des dizaines de petits tampons locaux au lieu d'un seul tampon global, ce qui rend plus difficile de voir si le PI dans son ensemble est en danger.
- **span_factor = 2 est une borne basse** : la loi de Kingman donne un lead-time qui croît hyperboliquement au-delà de 80 % d'utilisation [25] ; sur projets à variance forte, le coefficient sous-estime le lead-time réel.

11.3 Limites expérimentales

Sur *seed=42* le *gap* de capacité reste négatif sur toutes les compétences, donc le scénario HITL du retrait IA en goulot *data* ne se déclenche pas spontanément. Un mode *--amplify-bottleneck* ou un enrichissement des templates avec davantage de *features data* est nécessaire pour rendre la démonstration cliniquement déclenchable. Plus généralement, le dataset est synthétique : la validité externe est limitée et nécessitera une répllication sur des données réelles.

12 Points à explorer

Au-delà de la version livrée, six pistes d'extension sont identifiées. Elles sont volontairement techniques, chacune adresse un point précis de l'algorithme ou du système, mais on les annonce ici sans rentrer dans les détails d'implémentation. Le lecteur intéressé par les mécanismes sous-jacents peut se reporter à la section 3 (état de l'art).

12.1 P2 : Passage à NSGA-III pour mieux gérer 6 objectifs

En une phrase : remplacer NSGA-II par une variante plus récente, NSGA-III, mieux outillée pour comparer des plannings sur beaucoup de critères en même temps.

NSGA-II utilise la *crowding distance* pour préserver la diversité du front de Pareto. Cette technique est efficace jusqu'à trois objectifs, mais perd en précision au-delà. NSGA-III remplace ce mécanisme par un système de **points de référence** placés sur un hyperplan normalisé, qui guide explicitement la couverture du front [10, 12, 11]. À six objectifs comme nous en avons, le gain est documenté dans la littérature [7] ; le coût est un paramétrage supplémentaire (placement des points), qui demande sa propre étude.

12.2 P3 : NSGA-II + Q-Learning

En une phrase : laisser le système *apprendre* les bonnes dates de début, au lieu de les choisir par une

règle fixe.

[5] démontre qu'apprendre une politique de *timing* (choix du `start_day` de chaque activité) comme processus de décision markovien améliore les résultats par rapport à NSGA-II pur, PSO et ACO. C'est le complément naturel à notre seed *urgency* qui repose actuellement sur une heuristique fixe.

12.3 P4 : Données réelles Jira et multi-ART

En une phrase : brancher SAFESCALE sur les vraies données Jira et lui permettre de planifier plusieurs ART à la fois.

[17] adapte NSGA-II au multi-projet avec critères de durabilité, comblant un vide dans la littérature. Combiné à un adaptateur Jira Agile@Scale, ce travail permettrait le passage du prototype à un cadre industriel.

12.4 P5 : Project buffer global

En une phrase : remplacer les dizaines de petits tampons locaux par un seul gros tampon global de fin de PI, plus facile à piloter.

Consolider les *feeding buffers* locaux de DEC-018 en un *project buffer* global de fin de PI au sens de Goldratt 1997 (cité dans [18]) permettrait au RTE de piloter un seul tampon agrégé plutôt que des dizaines de buffers locaux. Le coût algorithmique est faible mais le changement de modèle mental côté utilisateur demande une étude.

12.5 P6 : Profils de ressource flexibles

En une phrase : laisser le système ajuster en cours de route combien de personnes travaillent sur une story, plutôt que de le figer à la planification.

[16] introduit MOFOA-HS pour les profils flexibles où la quantité de ressource par activité est déterminée en temps réel. Pertinent pour les ressources partagées entre équipes SAFe via les *component teams*.

12.6 P7 : Valoriser les équipes en sous-activité

En une phrase : quand une équipe a de la capacité libre, lui proposer d'anticiper des stories du PI suivant ou de venir en aide à une équipe surchargée.

Le retour terrain fait apparaître un cas symétrique de la surcapacité : des équipes engagées en dessous de leur potentiel sur un PI donné. Deux leviers sont à instrumenter dans l'outil : (i) proposer au RTE d'**anticiper des stories du PI suivant** compatibles avec les compétences disponibles de l'équipe sous-utilisée ; (ii) permettre à une équipe en sous-activité de **prendre en charge des stories d'autres équipes surchargées**, sous réserve de compatibilité de spécialisation. La modélisation des compétences (cf. section 4) et le mécanisme de retrait/ajout au step 3 posent déjà les briques nécessaires ; il reste à câbler les règles de décision et l'interface RTE pour ces deux mouvements.

13 Conclusion

Ce mémoire présente SAFESCALE, un prototype fonctionnel pour la planification SAFe PI Planning combinant NSGA-II à six objectifs, architecture multi-agents orchestrée et modèle de risque Critical Chain. L'apport principal n'est pas un nouvel algorithme mais

l'assemblage d'éléments déjà éprouvés ([4, 1, 23, 25]) dans le contexte précis du PI Planning SAFe, WSJF, sprints, équipes spécialisées, statut de *readiness*, dépendances inter-features, un contexte que l'entretien avec un Product Owner a aidé à cadrer.

Le tableau des décisions (table 2) relie chaque choix de conception à sa source, un article ou un point relevé lors de l'entretien PO. C'est un moyen, plus qu'un effet de style, de pouvoir justifier le système devant un jury.

À ce stade, le travail reste un prototype : le flow tourne de bout en bout sur le dataset de référence, mais l'évaluation chiffrée n'est pas encore faite (section 9). Les retours qualitatifs suggèrent que le découpage en cinq étapes avec HITL conditionnel rend les arbitrages lisibles ; il faudra le confirmer par des mesures. Les limites (section 11) et les pistes d'extension (section 12) indiquent les prochaines étapes vers un usage réel.

A Annexe : Glossaire des sigles

ACO

Ant Colony Optimization, optimisation par colonies de fourmis.

ART

Agile Release Train, regroupement de 5 à 12 équipes qui livrent ensemble.

BDI

Beliefs, Desires, Intentions, modèle d'agent autonome (non retenu ici).

CNP

Capacity Negotiation Protocol, étape 3 du flow, négociation de capacité.

CPO

Chief Product Owner, priorise les features de son périmètre.

DAG

Directed Acyclic Graph, graphe orienté sans cycle, modélise les dépendances.

HITL

Human-In-The-Loop, intervention d'un humain dans la boucle de décision.

LLM

Large Language Model, grand modèle de langage, utilisé ici comme traducteur.

MAS

Multi-Agent System, système multi-agents.

ML

Machine Learning, apprentissage automatique.

MOEA

Multi-Objective Evolutionary Algorithm, algorithme évolutif multi-objectifs.

MS-RCMPSP

Multi-Skill Resource-Constrained Multi-Project Scheduling Problem, extension multi-compétences et multi-projets du RCSP.

NSGA-II/III

Non-dominated Sorting Genetic Algorithm - algorithme génétique de tri non-dominé, versions 2 et 3.

PI

Program Increment, cycle de livraison de 8 à 12 semaines.

PO

Product Owner, porte le contenu d'une équipe.

RCSPSP

Resource-Constrained Project Scheduling Problem - problème d'ordonnement de projet sous contraintes de ressources.

ROAM

Resolved, Owned, Accepted, Mitigated, tableau SAFe de gestion des risques.

RTE

Release Train Engineer, facilitateur de l'ART.

SAFe

Scaled Agile Framework, cadre d'agilité à l'échelle.

SP

Story Points, unité d'estimation relative de l'effort.

WSJF

Weighted Shortest Job First, score de priorisation : valeur sur effort.

B Annexe : Glossaire des termes techniques

Optimisation et algorithmes évolutifs

Algorithme évolutif

Algorithme qui imite l'évolution naturelle (croisement, mutation, sélection) sur une population de solutions.

Métaheuristique

Méthode générale qui trouve de très bonnes solutions sans garantir l'optimum exact.

Individu / chromosome

Une solution candidate, ici, un planning complet.

Gène

Plus petit élément d'un chromosome ; ici, le couple (*story*, ressource).

Croisement

Combinaison de deux solutions parents pour produire un enfant.

Mutation

Modification aléatoire d'une partie d'une solution.

Élitisme

Conservation systématique des meilleures solutions d'une génération à l'autre.

Front de Pareto

Ensemble des meilleurs compromis : aucune solution du front ne peut être améliorée sur un critère sans en dégrader un autre.

Dominance

A domine B si A est au moins aussi bonne partout et strictement meilleure quelque part.

Crowding distance

Mesure favorisant des solutions bien réparties sur le front.

Recherche locale

Amélioration pas à pas d'une solution par petites modifications.

Algorithme mémétique

Algorithme évolutif augmenté d'une recherche locale.

Seed (initialisation)

Solution de départ intelligente injectée dans la population initiale.

Seed (graine aléatoire)

Nombre fixant le hasard ; à seed fixe, le programme est reproductible.

Distance de Hamming

Nombre de positions où deux solutions diffèrent.

NP-difficile

Problème dont aucun algorithme connu ne donne l'optimum en temps raisonnable à grande taille.

Ordonnancement

Makespan

Durée totale du projet, du début de la première story à la fin de la dernière.

Chemin critique

Plus longue chaîne de dépendances ; fixe la durée minimale du projet.

Tri topologique

Mise en ordre d'un DAG respectant toutes les dépendances.

Décodage

Transformation d'un chromosome en planning concret (dates de début/fin).

Tardivité pondérée

Retard d'une livraison multiplié par l'importance de l'élément en retard.

Goulot d'étranglement

Compétence trop demandée par rapport au nombre de personnes qui la maîtrisent.

SAFe et agilité

Epic / Feature / Story

Niveaux de découpage du travail ; la *story* est l'unité de planification.

Story point

Unité d'estimation relative de l'effort d'une story.

Vélocité

Story points réalisés par équipe et par sprint.

Poker planning

Estimation collective de l'effort des stories.

Readiness

Niveau de préparation d'une feature avant planification.

Sprint / cadence

Itération courte (3 semaines) ; la cadence est le rythme régulier des cérémonies.

Component team

Équipe transverse ; exception au principe « une équipe = une feature ».

Architecture et modélisation

Orchestration / émergence

Orchestration : un chef d'orchestre central pilote les agents. Émergence : agents autonomes sans chef.

HITL conditionnel

L'humain n'est sollicité que si un problème est détecté.

Mock

Fausse version d'un composant pour des tests reproductibles.

Durée-effort / durée-calendaire

L'effort réel (jours-personne) contre le temps réellement écoulé (lead-time).

Span factor

Coefficient transformant la durée-effort en durée-calendaire (pivot 2.0).

Loi de Kingman

Au-delà de ~80 % d'utilisation, les délais d'attente explosent, justifie le buffer 20 %.

Critical Chain

Méthode de Goldratt : regrouper les marges en buffers ciblés plutôt que gonfler chaque estimation.

Feeding buffer

Tampon de temps inséré après une story risquée.

Score de risque déterministe

Score 0-1 calculé par une formule fixe (mêmes entrées $\Rightarrow$ même score).

Évaluation

Dataset synthétique

Jeu de données généré artificiellement, aux dimensions calées sur un cas réel.

Reproductibilité

Propriété d'une expérience qui redonne le même résultat à chaque exécution.

Hypervolume

Volume de l'espace des objectifs couvert par un front de Pareto ; plus grand = meilleur.

Spread

Mesure de la bonne répartition des solutions sur le front.

Score composite

Note unique combinant les six objectifs normalisés et pondérés.

Dette technique

Travail de correction connu, reporté volontairement et documenté.

Références

- [1] X. Wen, Q. Song, Y. Qian *et al.*, « Effective Improved NSGA-II Algorithm for Multi-Objective IPPS », *Mathematics*, MDPI, 11(16) :3523, 2023.
<https://ideas.repec.org/a/gam/jmathe/v11y2023i16p3523-d1217570.html>
- [2] « An Improved NSGA-II Method for Solving Multi-objective Flexible Job-shop Scheduling Problems », *ResearchGate*, 2024.
<https://www.researchgate.net/publication/388139247>
- [3] J. Wang, L. Xu, S. Sun *et al.*, « Multi-objective optimization using improved NSGA-II for IPPS in a machining job shop for large-size valve », *PLoS ONE*, 19(6) :e0306024, 2024.
<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0306024>
- [4] « A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs », *Flexible Services and Manufacturing Journal*, Springer, 2024.
<https://link.springer.com/article/10.1007/s10696-023-09517-7>

- [5] « Bi-objective multi-mode resource-constrained multi-project scheduling using combined NSGA-II and Q-learning », *ScienceDirect*, 2024.
<https://www.sciencedirect.com/science/article/abs/pii/S156849462301219X>
- [6] « Is NSGA-II Ready for Large-Scale Multi-Objective Optimization? », MDPI, 2022.
<https://www.mdpi.com/2297-8747/27/6/103>
- [7] « Multi-objective optimization method for building energy-efficient design based on multi-agent-assisted NSGA-II », *Energy Informatics*, Springer, 2024.
<https://link.springer.com/article/10.1186/s42162-024-00394-4>
- [8] S. Verma, M. Pant, « A Comprehensive Review on NSGA-II for Multi-Objective Combinatorial Optimization Problems », IEEE Xplore, 2021.
<https://ieeexplore.ieee.org/document/9393947/>
- [9] Wang Yahui, Shi Ling, Zhang Cai *et al.*, « NSGA-II algorithm and application for multi-objective flexible workshop scheduling », *Sage Journals*, 2020.
<https://journals.sagepub.com/doi/full/10.1177/1748302620942467>
- [10] K. Deb, H. Jain, « An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Non-dominated Sorting Approach, Part I », *IEEE TEVC*, 18(4) :577-601, 2014.
<https://ieeexplore.ieee.org/document/6600851>
- [11] « Research on Multi-Objective Optimization of Project Scheduling Based on NSGA-III », *International Core Journal of Engineering*, 11(4), 2025.
<https://bcpublication.org/index.php/ICJE/article/view/8360>
- [12] V. H. S. Pham, N. T. P. Hoang, « Advancing multi-objective optimization in resource management using the NSGA-III algorithm », *Canadian Journal of Civil Engineering*, 52(7) : 1319-1335, 2025.
<https://cdnscepupub.com/doi/10.1139/cjce-2024-0298>
- [13] « A hybrid genetic algorithm for the resource-constrained project scheduling problem », *EJOR*, 185(2), 2008.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221707000616>
- [14] E. Goncharov, « A hybrid heuristic algorithm for the resource-constrained project scheduling problem », arXiv :2502.18330, 2025.
<https://arxiv.org/abs/2502.18330>
- [15] « Solving a real-life multi-skill resource-constrained multi-project scheduling problem », *Annals of Operations Research*, Springer, 2024.
<https://link.springer.com/article/10.1007/s10479-023-05784-7>
- [16] « Multi-Objective Multi-Skill Resource-Constrained Project Scheduling Considering Flexible Resource Profiles », MDPI *Applied Sciences*, 14(5), 2024.
<https://www.mdpi.com/2076-3417/14/5/1921>
- [17] Lostumbo, Bara, Iniesta, Calvet, « A NSGA-II Algorithm for Resource-Constrained Multi-Project Scheduling Problem Considering Sustainability Criteria », SSRN, 2024.
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5060061
- [18] « Fuzzy Resource Constrained Project Scheduling Optimization with Hybrid Multiobjective Genetic Algorithm », ACM ICBC, 2021.
<https://dl.acm.org/doi/10.1145/3448748.3448793>
- [19] « Multi-mode RCPSP with time-dependent resource costs and capacities : A bi-objective approach », *ScienceDirect*, 2025.
<https://www.sciencedirect.com/science/article/pii/S0957417425045701>
- [20] R. Pellerin *et al.*, « A survey of hybrid metaheuristics for the resource-constrained project scheduling problem », *EJOR*, 280(2), 2020.
<https://www.sciencedirect.com/science/article/abs/pii/S0377221719300475>
- [21] W. Herroelen, B. De Reyck, E. Demeulemeester, « Resource-constrained project scheduling : A survey of recent developments », *Computers & Operations Research*, 25(4), 1998.
<https://www.sciencedirect.com/science/article/abs/pii/S0305054897000551>
- [22] Scaled Agile Inc., *SAFe 6.0 Framework, PI Planning*.
<https://framework.scaledagile.com/pi-planning>
- [23] « Enhancing multi-objective evolutionary algorithms with machine learning for scheduling problems : recent advances and survey », *Frontiers in Industrial Engineering*, 2024.
<https://www.frontiersin.org/journals/industrial-engineering/articles/10.3389/fieng.2024.1337174/full>
- [24] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, « A Fast and Elitist Multiobjective Genetic Algorithm : NSGA-II », *IEEE Transactions on Evolutionary Computation*, 6(2) :182-197, 2002.
<https://ieeexplore.ieee.org/document/996017>
- [25] D. G. Reinertsen, *The Principles of Product Development Flow : Second Generation Lean Product Development*, Celeritas Publishing, 2009.
https://www.researchgate.net/publication/235268824_The_Principles_of_Product_Development_Flow
- [26] J. Blazewicz, J. K. Lenstra, A. H. G. Rinnooy Kan, « Scheduling subject to resource constraints : classification and complexity », *Discrete Applied Mathematics*, 5 :11-24, 1983.
<https://www.sciencedirect.com/science/article/pii/0166218X83900124>